

Unless otherwise stated, copyright © 2025 The Open University, all rights reserved.

Printable page generated Saturday, 5 July 2025, 12:36

TMA 02

This tutor-marked assignment (**TM111 TMA 02**) must be submitted by 12 noon (UK local time) on **17 July 2025**.

This module requires all assignments to be submitted electronically. To submit an assignment, please follow the link(s) from your StudentHome page to the online TMA/EMA service.

If you foresee any difficulty with submitting your assignment on time, you should contact your tutor well in advance of the cut-off date.

Important – help with assignment questions

You should read the *TM111 Assessment guide* in the Assessment section of the TM111 website before answering the questions here.

All written answers must be submitted in a word-processor document that is compatible with Microsoft Word, that is, with one of the following file extensions: .doc, .docx or .rtf. See the section on 'What to submit for TMAs' in the *TM111 Assessment guide*.

If you need further guidance on any assignment questions, please ask your tutor, who will be happy to give you advice.

Note that screenshots should show just the scripts concerned, not the whole OUBuild window or your entire screen. Refer to the *TM111 Assessment guide* for further information on screenshots.

Reminder: in order to pass the module, you will need to gain a weighted average of 40% or more for your TMAs and iCMAs, and a mark of at least 30% for TMA 03. The weighted average of your iCMA and TMA scores will give your overall module score.

Questions

In this TMA, you will work on some prototype programs to assist a teacher working with a class of young pupils.

In each question, unless directed otherwise:

- you should assume that the user enters valid data, and you do not need to test your program with invalid inputs
- when designing an algorithm or creating code, you should do no more than the question asks – do not add extra features.

Marks may be deducted if the specification isn't followed exactly.

You must provide a word count for each question or question part where a limit is specified.

Question 1 (20 marks)

This question focuses on Part 1 (variables), Part 2 (constants, lists, arithmetic and joining strings) and Part 3 (selection).

Open the project TM111_02_Q1.sb2, which can be downloaded from the OUBuild software and project files area of the module website – see 'TMA 02 OUBuild software projects'.

This project is intended to implement a program to help the teacher record the number of birds recently counted by the pupils on a nature walk.

Before recording each pupil's score, the teacher will start the program using the green flag. She will then press her space key whenever she wants to record a pupil's bird count, which is a whole number out of a maximum of 80. If the number, as a percentage of the maximum of 80, is more than 85%, then the pupil will be awarded a Wren sticker.

In this project, we have provided a `when_green_flag_clicked` script and a `when[space]key_pressed` script. Consider these scripts carefully, and then answer the following questions:

- a. Imagine a scenario in which the user starts the program with the green flag, then presses the space key, enters 'Keon', and then 70 when prompted.

Copy and complete the following step-by-step description of what happens, starting from the point at which the user is prompted for input.

The user is asked to enter a pupil's name. Their input, 'Keon', is stored in the variable name.

The user is asked to enter the score achieved by the pupil out of a maximum of 80...

Your description should make clear what data is stored in the variables and the list involved, and the result of any comparison that is made.

You should describe what happens in the particular scenario described here, with the inputs 'Keon' and 70, not what the script does in general or what might have happened with different inputs.

(4 marks)

b.

- i. Identify a numerical value used in this program that might appropriately be stored in a constant. (Recall that a constant is a value that plays a significant role in a program and does not change during the time the program is running.) There may be more than one possibility but you are only required to identify one. You are not asked to implement this constant.

(1 mark)

- ii. What might be an appropriate name for the constant you have chosen?

(1 mark)

c. Amend the `when_green_flag_clicked` and `when[space]key_pressed` scripts so that:

- If a pupil achieves a score greater than 85% of the maximum, their name is added to the list `wren_list`.
- Otherwise, their name is added to a list `magpie_list` (which you should create and initialise appropriately).

Take a screenshot of your resulting scripts, and paste it into your TMA document.

(5 marks)

d.

- i. Further amend the `when_green_flag_clicked` and `when[space]key_pressed` scripts so that:
 - If a pupil achieves a score greater than 85% of the maximum, their name is added to the list `wren_list`.
 - If a pupil achieves a score less than 40% of the maximum, their name is added to a list `robin_list` (which you should create and initialise appropriately).
 - Otherwise, their name is added to the list `magpie_list`.

Check that your program passes the following tests. (You aren't expected to give any details of your testing; this is just to help you check your program.)

Test number	Test purpose	Test inputs		Expected results
		Score achieved, out of 80	Name	List name is added to
1	Just below lower boundary value	31	p1	robin_list
2	Lower boundary value	32 (equivalent to 40%)	p2	magpie_list
3	Just above lower boundary value	33	p3	magpie_list
4	Just below upper boundary value	67	p4	magpie_list
5	Upper boundary value	68 (equivalent to 85%)	p5	magpie_list
6	Just above upper boundary value	69	p6	wren_list

Take a screenshot of your resulting scripts, and paste it into your TMA document.

(7 marks)

- ii. Different forms of selection structure could be used to meet the specification in Question 1(d)(i). Briefly describe an alternative form of selection structure to the one you chose.

(2 marks)

Save your final version of the OUBuild project for Question 1 and submit it as *TM111_02_Q1_PI.sb2* (where *PI* is your OU personal identifier, e.g. A1234567) in your TMA zip file. Alternatively, you may use your OUCU instead of your *PI*.

Question 2 (20 marks)

This question focuses on Part 2 (arithmetic, rounding, joining strings, custom blocks) and Part 3 (selection). It also involves changing a sprite's size, introduced in Activity 4.2 of Part 4.

Open the project *TM111_02_Q2.sb2*.

This project is intended to provide a program to help the pupils practise calculating percentages. It contains a sprite named Flower, equipped with three costumes, neutral, smile and frown. It also contains a custom block, *shrink*.

Start by exploring the *shrink* block.

Then create a program to achieve the following.

- When the user presses their space key Flower should wear the costume neutral, with its size set to 100%. The user should be asked to calculate a percentage, to the nearest whole number, in the format shown in Figure 1.

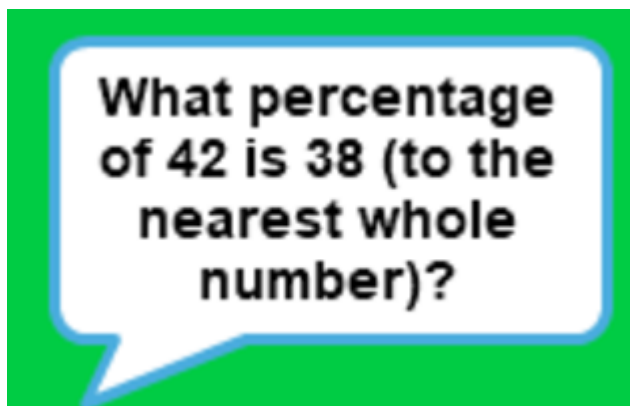


Figure 1 Example question in OUBuild

Show description ▼

Both numbers should be whole numbers chosen at random. The first number (42, in the above example) should be between 1 and 100 inclusive. The second number (38, in the above example) should be between 1 and the first number inclusive.

If the user gets the answer correct they should be told 'Right' and the flower should 'expand' to congratulate them. If they get the answer incorrect, they should be told so and given the correct answer, rounded, for example 'Wrong. The answer is 90', and the flower should 'shrink' in commiseration.

Your program should use the following:

- The custom block shrink to make the flower shrink. You should not change the definition of this block.
- A custom block expand that you should create and define to make the flower 'expand'. There is no strict specification for expanding, but your block should involve the following:
 - playing some form of music or sound, which you should bring into the project using any of the 'New sound:' facilities available in the Sounds pane
 - the costume smile
 - the sprite visibly increasing in size.

We encourage you to design an algorithm before starting to create your program, and to test your program; however, for this question you are not required to submit anything other than the complete program.

Be sure to test your program in situations where the user gets the answer correct and in situations where they get the answer incorrect.

Marks will be awarded for the following:

- a. Forming and displaying an appropriate percentage calculation question.

(8 marks)

- b. Obtaining user input and taking the appropriate actions.

(12 marks)

Take a screenshot of your scripts and paste it into your TMA document.

Save your OUBuild project for Question 2 and submit it as TM111_02_Q2_Pi.sb2 (where Pi is your OU personal identifier, e.g. A1234567) in your TMA zip file. Alternatively, you may use your OUCU instead of your Pi.

Question 3 (25 marks)

This question focuses on Part 3 (selection) and Part 4 (repetition). As inspiration for this question, you might like to revisit Activity 4.8 of Part 4, which uses a list of specific characters.

The teacher wishes to see if her pupils can make sense of words in which certain letters have been removed.

She requires a `when[space]key_pressed` script that will take a word from the user and display a new word that is the same as the original except that each letter from the first ten letters of the alphabet has been removed. For clarification, the first ten letters of the alphabet should be considered as: a, b, c, d, e, f, g, h, i and j.

The user should input the word as a single string and the new word that is displayed should be a single string.

For example, if the user enters

'party'

then

'prty'

is displayed.

One way to create the new word to be displayed is to start with the new word as an empty string and then work through the letters in the original word from first to last, creating the new word incrementally. If the current letter is not in the first ten letters of the alphabet, then we add it to the new word, otherwise we do not add it to the new word. This might be visualised as in Figure 2.

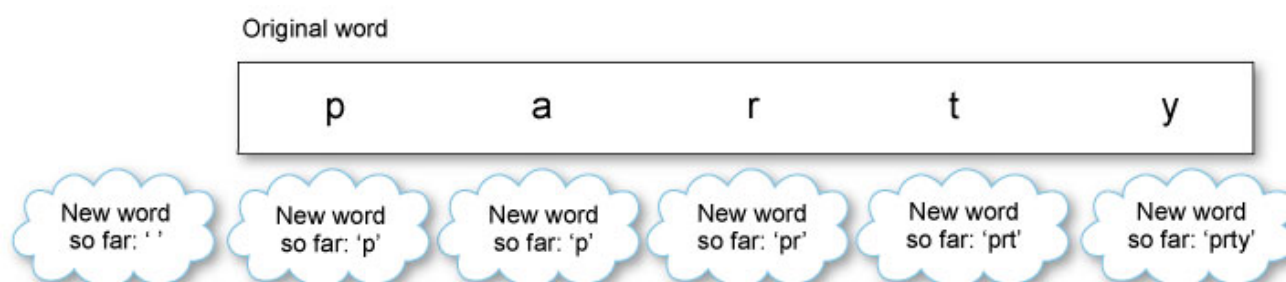


Figure 2 Example of how to solve the problem described

Show description ▼

Recall that, as in Part 4 Activity 4.8, a simple way to check whether a given character is one of a set of specific characters is to check whether it is in a list that has been populated with those specific characters.

- a. Create and write down an algorithm to solve this problem. You might like to use the idea above, or an alternative of your own.

(10 marks)

- b. Create a `when[space]key_pressed` script to implement your algorithm. Depending on your algorithm and the way in which you choose to implement it, you may also decide to have a `when_green_flag_clicked` script.

Take a screenshot of your script(s) and paste it into your TMA document.

(11 marks)

- c. Copy the following table into your TMA document, and add to it two tests you would perform to check whether the completed program fulfils the specification. (Several tests might be appropriate; however, you are only required to add two.)

Test number	Test purpose	Inputs	Expected results
		Word	New word displayed
1	First and last letters are in the first ten letters of the alphabet	apple	ppl
2			

3

(4 marks)

Save your OUBuild project for Question 3 and submit it as *TM111_02_Q3_PI.sb2* (where *PI* is your OU personal identifier, e.g. A1234567) in your TMA zip file. Alternatively, you may use your OUCU instead of your *PI*.

Question 4 (25 marks)

This question focuses on Part 4 (repetition), Part 5 (components) and Part 6 (sorting).

The teacher runs an after-school computing club and stores the names of children who belong to this club in a list, which at present is unsorted.

She is considering ways to sort the list and is interested in a sorting algorithm she has recently heard about. The algorithm, which is new to her, is a modified version of selection sort, sometimes called ‘double selection sort’. She explains it to you.

‘It’ll make it easier to describe’, she begins, ‘if I use the terms “less than” to mean “comes before in alphabetical order” and “greater than” to mean “comes after in alphabetical order” (recall this is how `[] < []` and `[] > []` work in OUBuild).’

She continues, ‘In traditional not-in-place selection sort we start with an unsorted list of names and create a new list which will eventually hold the names in sorted order.

We repeatedly search the unsorted list for the name that is the least in the list and when it is found we move it to the end of the new list.

On each pass the unsorted list shrinks by one name and the sorted list grows by one. Eventually the unsorted list becomes empty, and the new list contains all the names, now sorted in alphabetical order’.

You saw in Part 6 Activity 6.15 that with a list of n items the first pass of selection sort needs $n - 1$ comparisons, the second pass $n - 2$, and so on, with the last pass needing just one comparison. Altogether this comes to about $(n - 1)^2 / 2$ comparisons. The exact number is

$$n \times (n - 1) / 2$$

Double selection tries to reduce the number of comparisons. It starts with *two* empty lists, let’s call them ‘least’ and ‘greatest’. On each pass we search for both the name that is least *and* the name that is greatest. The least name is moved to the end of the ‘least’ list and the name that is greatest is moved to the front of the ‘greatest’ list.

Two names are moved on each pass, so only half as many passes are needed. When all the names have been moved to the new lists, the names in both will be sorted. The names in 'greatest' will all come after the names in 'least', so we can just push these lists together to get the final sorted list.

Here's an example. 'I've used just letters instead of names, to save space'.

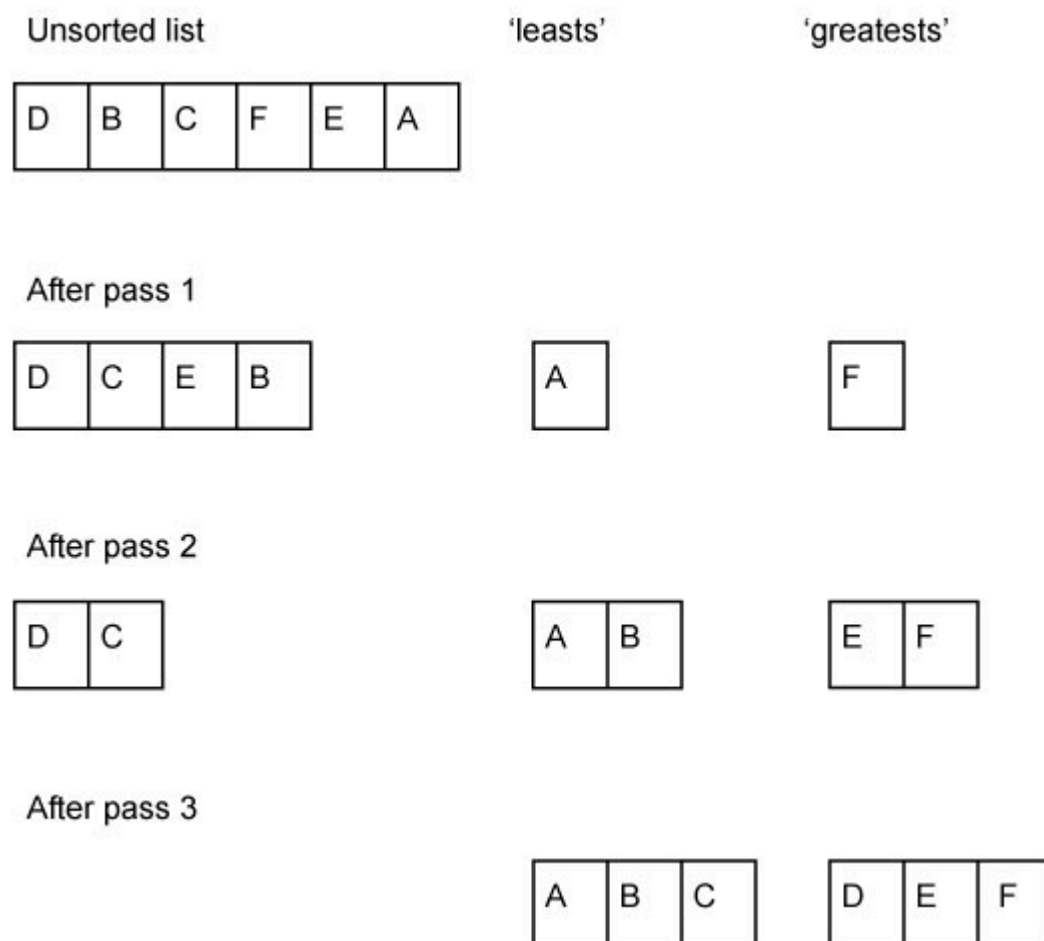


Figure 3 Double selection sort

Show description ▼

'But how does this help?' you ask. 'Won't the overall number of comparisons be identical? Half as many passes, yes, but twice the number of comparisons per pass, so no reduction'.

'That's the clever bit', she replies. 'There is a way to do the comparisons more efficiently'.

Imagine we are in the middle of a pass and know the least name so far and the greatest name so far. Suppose the next pair of names we need to consider are name1 and name2.

We need to decide whether we have found a new value for the least name so far. Similarly, we need to decide if we have a new greatest name so far. One way to do this is shown in Figure 4.

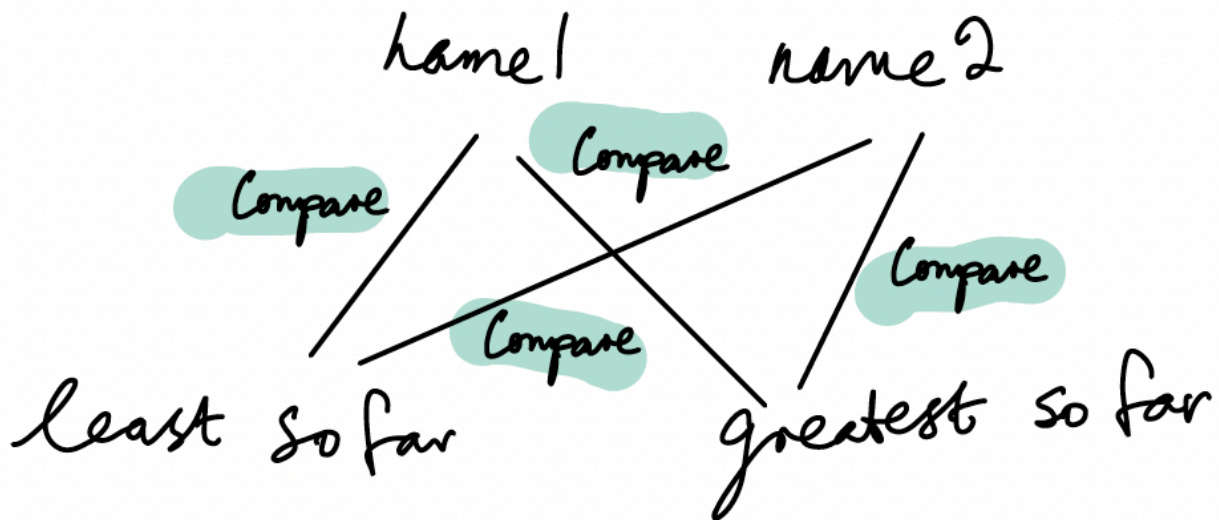


Figure 4 Name comparison 1

Show description ▼

If we use this approach, there will be four comparisons which will indeed offer no improvement over traditional selection sort. However, a better approach is illustrated in Figure 5, which only requires three comparisons!

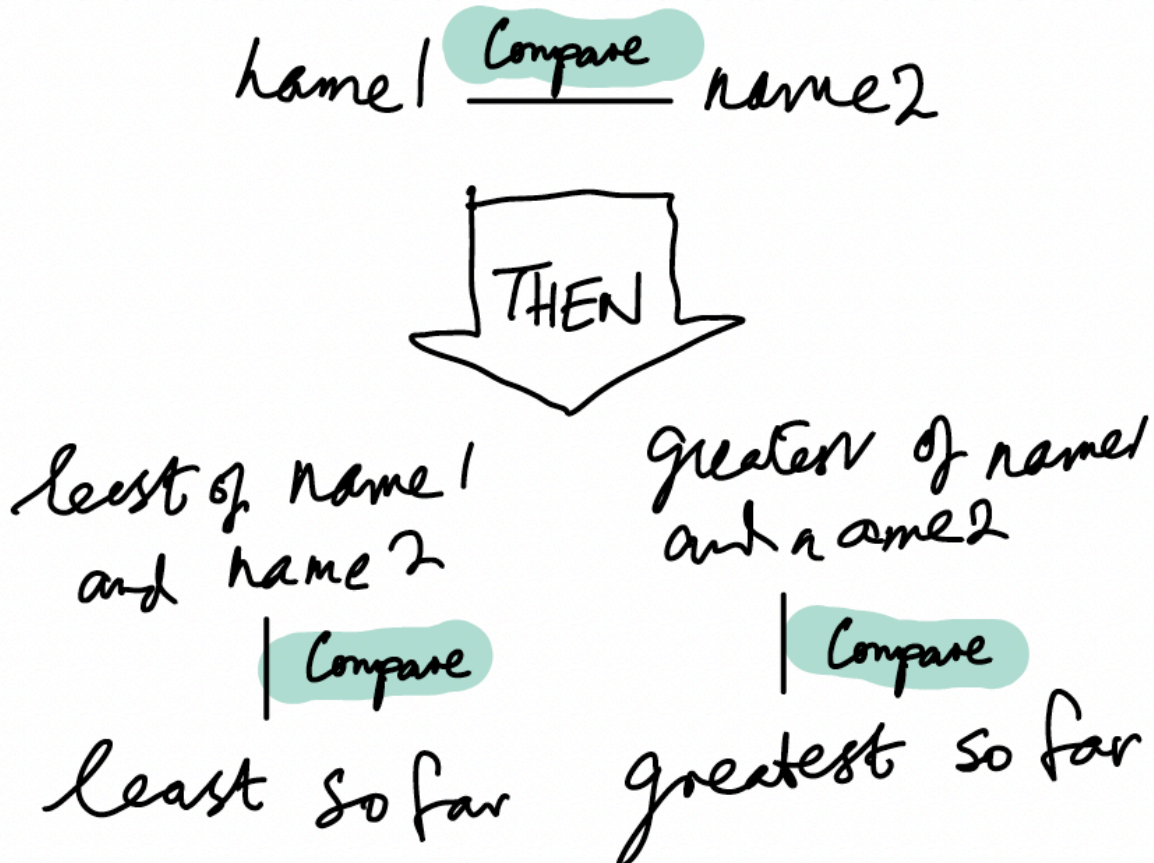


Figure 5 Name comparison 2

[Show description ▼](#)

Using this method means double selection sort should only need $\frac{3}{4}$ as many comparisons as traditional selection sort, a saving of 25%.

The teacher has written a large part of an OUBuild program to try double selection sort out, but has been called away on another project, and has asked you to write the remainder of the program and carry out some experiments.

Now open the project `TM111_02_Q4.sb2`. It contains:

- A list `names`, which initially contains 10 letters arranged in random order. (Note that for simplicity we are using letters rather than actual names and assume that no two letters are the same.)
- Two lists `mins` and `maxs`. These are the lists the least and greatest names identified on each pass will be moved to.
- A component, the custom block `least_and_greatest`, which uses the method described by the teacher to find the position in `names` of the least and the greatest names. When these are found they are stored in two variables `least` and `greatest` respectively. Note that these are the *positions* of the least and greatest items, not the items themselves.

There is no need for you to explore the inner workings of this component, which has been provided precisely so we don't need to get bogged down in the detail. *Note particularly that you should on no account make any changes to the component.*

- A second component, the custom block `delete_least_and_greatest`. The reason we need this is that once we have copied the least and greatest names to the new lists, we want to delete them from the original list, and the order of deletion matters, because when we delete an item from a list it affects the positions of all the items further along. The component provides a way of doing the deletions we need without worrying about this complication.
- Variables `pos`, `mn` and `mx`. These are used exclusively by the `least_and_greatest` component, and you will not need to use them directly.
- A variable `comparison_count` which you will use to count the number of comparisons required to sort the list `names`.

- a. Your first task is to test the `least_and_greatest` custom block. Drag a block `least_and_greatest` from the More Blocks palette to the Scripts pane.

Make sure that watchers for `least` and `greatest` are visible on the stage, and click on this block to run it and you should see that `least` and `greatest` are respectively 1 and 3. You can check that these are correct; the letter at position 1 is 'B', which is the alphabetically least item in the list, and that at position 3 is 'X', the alphabetically greatest.

Below we have started a table, similar to Table 5.1 in Part 5 and filled in the first row, corresponding to the test above.

Test no.	Test purpose	Existing data	Expected results	
		names	least	greatest
1	Least name at position 1, greatest at position 3.	B I X E R Q W V D S	1	3
2				
3				

Copy this table and add to it two more tests that you would perform to check that the component fulfils its specification. You do not have to start with the same list contents each time. If you want to restore the original state of the list at any point, you can do so by importing the contents of `names1.txt`, following the procedure described on page 69 of Block 2 Part 4.

Carry out your tests to confirm the results are as expected.

Copy your table into your TMA document. You do not need to include screenshots of your tests.

(2 marks)

- b. Next you will complete a `when_green_flag_clicked` script that we have started. When complete your script should implement the algorithm below, which will sort the list names using double selection sort. The script must use the custom blocks `least_and_greatest` and `delete_least_and_greatest`, and the variables `least` and `greatest`, where indicated by the algorithm.

Repeat until length of list names equals zero
Get positions of least and greatest names
Append name at position least at end of list mins
Insert name at position greatest at beginning of list maxs
Delete names at least and greatest
Repeat until length of list maxs is zero
Append item at position 1 of maxs to end of mins
Delete item at position 1 of maxs

Show description ▼

At the end the final sorted list will be in `mins`.

Re-import the list of names if necessary and run your script. If everything is right, then at the end the list `mins` should hold the 10 names in alphabetical order.

Take a screenshot of your `when_green_flag_clicked` script and paste it into your TMA document.

(10 marks)

- c. In this part you will investigate how many comparisons between list items double selection sort requires for lists of different lengths.

- i. To count the comparisons, we have provided a variable `comparison_count`.

The component automatically increments this count by 1 for each comparison made.

However we also need to initialise `comparison_count` to 0 at the start of the `when_green_flag_clicked` script, before we start the sort. Add a block to initialise `comparison_count` at an appropriate point in the script.

Then reload the names list, make sure the watcher for `comparison_count` is visible on the stage, and run the `when_green_flag_clicked` script.

You should find that once the script has run the value of `comparison_count` is 35.

Take a screenshot of your modified `when_green_flag_clicked` script and paste it into your TMA document.

(1 mark)

- ii. As well as the file `names1.txt` which contains a list of 10 letters we have also provided `names2.txt` and `names3.txt`, which contain 20 and 30 random entries respectively. For simplicity we wanted to avoid any item being duplicated in a list, so we have used pairs of letters such as AB or DQ this time.

Import `names2.txt` and run your script, noting how many comparisons the sort takes. Then repeat the process using `names3.txt`.

Copy and complete the following table.

Size of list	10	20	30
Number of comparisons	35		

(2 marks)

- iii. Does the number of comparisons grow in a linear manner? Briefly justify your answer.

(2 marks)

- iv. The teacher reminded us earlier that traditional selection sort takes exactly $n \times (n - 1) / 2$ comparisons and we have seen that we expect double selection sort to take about 25% fewer, i.e. 0.75 of the number for traditional selection sort.

Copy and complete the following table

Size of list (n)	10	20	30
$n \times (n - 1) / 2$			

$0.75 \times n \times (n - 1) / 2,$ rounded up to next whole number			
--	--	--	--

Compare your answers in the third row with the actual numbers of comparisons you recorded in Part ii. above. Comment briefly on what you find and suggest a rough estimate for how many comparisons would be required if we were sorting 1,000 names.

(6 marks)

- d. Figure 6.40 of Block 2 Part 6 graphs the performance of some different sorting algorithms. Briefly describe where you would expect double selection sort to sit in this graph. You are not asked to draw anything, just to answer in words.

(2 marks)

Save your final OUBuild project for Question 4 and submit it as *TM111_02_Q4_PI.sb2* (where *PI* is your OU personal identifier, e.g. *A1234567*) in your TMA zip file. Alternatively, you may use your OUCU instead of your *PI*.

Question 5 (10 marks)

What you should have already completed?

You should have completed your 'Personal development action plan' form as part of Activity 6.21 (online) in Block 1 Section 6.6.

Answering this question

Include a copy of your completed 'Personal development action plan' form in your TMA 02 zip file.

Note: this is not the same as the PDP self-evaluation form that you submitted in TMA 01. Do not resubmit your PDP self-evaluation form.

Marks

You will be awarded up to 10 marks for reflecting on and engaging with the planning process as specified in Activity 6.21 and submitting your completed 'Personal development action plan' form as part of TMA 02.

TMA 02 Checklist

Have I ...	Yes	No, what should I do?
read the <i>TM111 Assessment guide</i> ?		This is on the Assessment tab of the module website, where you accessed the TMA.
have I read and taken note of the feedback on TMA01?		Feedback will have been received on your TMA01 documents. Make sure you follow the link to download your feedback and read the overall summary as well as the detailed comments on your work.
saved my TMA document as a word-processed document?		TMA documents should be produced as a word-processed document that can be opened with Microsoft Word. Acceptable file extensions are .doc, .docx or .rtf.
saved my OUBuild files with the correct names?		All OUBuild files should have a .sb2 extension. There should be one file for each question with the naming convention TM111_02_Qx_PI.sb2 where the x is replaced with the question number and the PI with your OU personal identifier. Alternatively, you may use your OUCU instead of your PI.
completed my PDP action plan?		Complete your Personal development action plan, which is part of Activity 6.21 in Block 1 Section 6.6. Note: this is not the same as the PDP that you submitted in TMA 01. Do not resubmit your PDP.
included the necessary files in the zip file?		Ensure that you are submitting: • TMA document • OUBuild files for Questions 1–4 • Personal development action plan.
ensured that the zip file is in the correct format and has a .zip extension?		Please make sure that the zipped file has the .zip extension (and not others such as .rar or .7zip) so the online TMA handler can extract your files properly. More help can be found in the <i>TM111 Assessment guide</i> .